NAME: BISMA FATIMA

CLASS: CS A

ROLL NO: F24605036

COURSE: SOFTWARE ENGINEERING

COURSE CODE: CS-250

LECTURER: MA'AM SABA FAROOQ

ASSIGNMEANT 04

Q1: Explain the concept of software evolution and discuss why it is inevitable in modern software systems.

Software Evolution

Software evolution means the continous modification of software after it has been developed and released. It includes fixing bugs, adding new features, improving performance and updating the system according to changing user and business needs. Software systems do not remain same for a long time and keep changing during their life cycle.

Why Software Evolution is Inevitable

Software evolution is unavoidable because:

- User requirments keep changing with time
- Technology changes rapidly with new hardware and operating systems
- Business rules and laws are updated frequently
- Errors and bugs are discovered after deployment
- Software must compete in market to stay useful

If software does not evolve, it becomes outdated and loses its usefulness.

Q2: Describe Lehman's Laws of Software Evolution and explain their significance in software maintenance.

Lehman's Laws of Software Evolution

Lehman proposed different laws to explain how large software systems evolve over time.

1. Law of Continuing Change

- **Principle**: An E-type system must continuously change or it becomes less useful.

- **Significance**: If software does not evolve with its environment, it loses relevance and eventually becomes obsolete.

2. Law of Increasing Complexity

- **Principle**: As a system evolves, its complexity increases unless deliberate efforts are made to reduce it.
- **Significance**: Without refactoring and redesign, technical debt accumulates, making systems harder to maintain.

3. Law of Self-Regulation

- **Principle**: Evolution is a self-regulating process influenced by feedback from users, developers, and management.
- **Significance**: Growth patterns become predictable, aiding in long-term planning and resource allocation.

4. Law of Conservation of Organizational Stability

- **Principle**: The overall rate of software development remains stable over time, regardless of added resources.
- **Significance**: Simply adding more developers does not accelerate progress; improving processes and architecture is more effective.

5. Law of Conservation of Familiarity

- **Principle**: Users and developers must retain familiarity with the system. Too much change reduces usability and increases errors.
- **Significance**: Evolution must be gradual, balancing innovation with stability.

6. Law of Continuing Growth

- **Principle**: Systems must continually grow in functionality to satisfy users.

- **Significance**: User expectations and competitive pressures demand ongoing feature expansion.

7. Law of Declining Quality

- **Principle**: The perceived quality of a system declines unless it is rigorously maintained and adapted.
- **Significance**: Even if functionally correct, outdated systems feel "low quality" compared to modern standards, requiring preventive maintenance.

Significance of Lehman's Laws in Software Maintenance

Lehman's Laws provide a **framework for understanding why maintenance is not just fixing bugs but a strategic activity**:

- They explain why **continuous investment** in software is necessary.
- They highlight the importance of **refactoring, preventive maintenance, and reengineering** to control complexity and technical debt.
- They show that **quality is a moving target**—systems must evolve to meet rising expectations.
- They emphasize that **evolution is predictable** in many ways, allowing organizations to plan resources and processes effectively.